Volatility Term Structure Forecasting
for Dynamic Portfolio Optimization

Programming in Finance II — Big Project 2026
Università della Svizzera italiana

Luca Anselmi Stefan Vidovic Arnel Hodza

June 2026

Abstract

This paper documents the design, implementation, and empirical results of a volatility-based dynamic portfolio optimization system. Rather than predicting stock price directions — near-impossible under the efficient market hypothesis — we predict the volatility term structure of ten Dow Jones 30 stocks using the Nelson-Siegel model and XGBoost. Predicted volatility parameters drive three distinct portfolio allocation methods. A walk-forward backtest from 2018 to 2026 compares six strategies on Sharpe ratio, maximum drawdown, CAGR, and stress-period performance. The system is implemented in Python with an agentic pipeline on GitHub and a Streamlit dashboard with AI-generated insights via the Anthropic Claude API.
GitHub: https://github.com/vidovsusi-sys/volatility-trading-agent

Contents

1 Project Plan

1.1 Motivation

The intuitive goal of any market participant is to predict whether a stock will rise or fall. The fundamental difficulty is that daily returns exhibit autocorrelation close to zero — the efficient market hypothesis [Fama, 1970] implies that public information is already embedded in prices instantaneously. Volatility, by contrast, has memory: turbulent days tend to be followed by turbulent days. This volatility clustering, formalized by Engle [1982] (Nobel Prize 2003) and extended by Bollerslev [1986], is one of the most robust stylized facts in empirical finance.

Our project exploits this property. Instead of forecasting returns, we forecast the entire volatility term structure of each stock and use those forecasts to dynamically reweight a portfolio every day — giving less weight to stocks we expect to be more volatile tomorrow. The Variance Risk Premium [Carr and Wu, 2009], computed as the difference between implied volatility (VIX) and realized volatility, serves as an additional macroeconomic signal.

1.2 Research Questions

The project is structured around four empirical research questions:

RQ1 Does XGBoost outperform ARMA in forecasting Nelson-Siegel betas, both in forecast accuracy and in portfolio performance?

RQ2 Does ML-based volatility forecasting (Method A, XGBoost) beat Historical Risk Parity, which uses only past realized volatility?

RQ3 Does incorporating the full shape of the volatility curve (Method B: level, slope, curvature) add value over the level alone (Method A)?

RQ4 Does combining a momentum signal with predicted volatility (Method C) further improve risk-adjusted performance?

1.3 Architecture and Scope

The pipeline has seven sequential stages: (1) data acquisition via `yfinance` for 10 stocks, VIX, and S&P500 from July 2014 to present; (2) realized volatility across six horizons; (3) Nelson-Siegel fitting, compressing 60 volatility series into 30 beta parameters per day; (4) Variance Risk Premium as exogenous forecasting input; (5) walk-forward XGBoost and ARMA forecasting; (6) three portfolio optimization methods with weight constraints; (7)

backtesting across six strategies with realistic transaction costs.

The project satisfies the 2026 generic criteria with two advanced components: **machine learning** (XGBoost walk-forward forecasting) and **large language model** (Claude API for AI Insights in the dashboard), plus advanced data visualization (interactive Streamlit dashboard) and a web frontend.

2 Project Diary

Development proceeded over eight weeks with regular commits to GitHub and iterative use of AI tools throughout.

Problem definition and data. The team began by defining the research questions and selecting 10 stocks from the Dow Jones 30 by minimizing average pairwise correlation, subject to liquidity and data availability constraints. The selected universe — CRM, VZ, WMT, INTC, UNH, MRK, BA, NKE, CVX, AMGN — provides good sectoral diversification while keeping the pipeline manageable. All raw data is saved immutably to `data/raw/` to guarantee reproducibility across runs.

Volatility pipeline. Rolling realized volatility was computed across six horizons and fitted daily with the Nelson-Siegel model using fixed $\lambda = 0.04$ following Diebold and Li [2006]. The Variance Risk Premium was implemented following Carr and Wu [2009] and validated against the known stylized fact that VRP is positive on average and negative during market crises.

Forecasting and portfolio construction. The walk-forward forecasting loop was designed with strict no-look-ahead rules: normalization uses only historically available data, a 5-day gap separates training from prediction, and a minimum of 500 training days is required. Both XGBoost and ARMA models were implemented and compared. Three portfolio methods translate predicted betas into daily weights via constrained optimization.

Dashboard and agents. The Streamlit dashboard was built with tabs for equity curves, stress analysis, portfolio weights, and an AI Insights tab that calls the Anthropic Claude API to automatically interpret backtesting results. Two agents automate the pipeline: a Data Agent that downloads, validates, and commits updated data to GitHub, and a Trading Agent that runs the full pipeline end-to-end and opens a Pull Request with results — satisfying the course requirement of at least one agent-made PR.

Tools used. Python 3.13 with `xgboost`, `statsmodels`, `streamlit`, `yfinance`, `scipy`, `plotly`, and `anthropic`. Claude (Anthropic) was used throughout the project for methodology design, code generation, debugging, and documentation, and is acknowledged as the primary AI tool per course requirements.

3 Methodology

3.1 Realized Volatility

Log returns are defined as $r_t = \log(P_t/P_{t-1})$. Annualized realized volatility over horizon N is:

$$\sigma_i(t, N) = \sqrt{252} \cdot \text{std}(r_{t-N+1}, \ldots, r_t) \tag{1}$$

Six horizons $N \in \{5, 10, 21, 42, 63, 126\}$ span one week to six months, capturing short-term panic, quarterly earnings cycles, and long-run structural risk. This produces 60 annualized volatility series per day across 10 stocks.

3.2 Nelson-Siegel Term Structure Model

The Nelson-Siegel model [Nelson and Siegel, 1987] represents the volatility term structure as:

$$\sigma(\tau) = \beta_0 + \beta_1 \cdot f_1(\tau) + \beta_2 \cdot f_2(\tau) \tag{2}$$

where the factor loadings with fixed $\lambda = 0.04$ are:

$$f_1(\tau) = \frac{1 - e^{-\lambda\tau}}{\lambda\tau}, \qquad f_2(\tau) = f_1(\tau) - e^{-\lambda\tau} \tag{3}$$

The three parameters carry precise economic meaning: β_0 is the long-run volatility level (as $\tau \to \infty$, $\sigma \to \beta_0$), β_1 is the slope (short-term minus long-term volatility), and β_2 is the curvature (height of the medium-term hump, peaking near $\tau \approx 25$ days). Daily OLS estimation reduces 60 noisy series to 30 economically interpretable parameters.

3.3 Variance Risk Premium

Following Carr and Wu [2009]:

$$\text{VRP}_t = \text{VIX}_t - \sigma_{\text{S\&P500}}(t,\, 21) \tag{4}$$

Historically positive on average (mean 3.47% in our sample), VRP reflects investors paying a premium to hedge uncertainty. It enters the forecasting models as an exogenous macroeconomic signal alongside lagged betas.

3.4 Walk-Forward Forecasting

ARMA benchmark. For each of the 30 beta series, an ARMA(p, q) model is fitted with order selected by BIC minimization. Predictions use only data available up to the previous day.

XGBoost. Chen and Guestrin [2016] builds an ensemble of gradient-boosted decision trees with no prior structural assumption, capable of capturing non-linear relationships and regime changes that ARMA cannot detect. The feature matrix contains lagged betas at lags 1, 2, and 5 days for all stocks (90 features) plus lagged VRP (3 features). A separate model is trained per beta series and per walk-forward window, yielding 270 total models. Predictions begin in 2018 after sufficient training history accumulates.

3.5 Portfolio Optimization

Method A — Risk Parity with predicted β_0:

$$w_i^A = \frac{1/\hat{\beta}_{0,i}}{\sum_j 1/\hat{\beta}_{0,j}} \tag{5}$$

Stocks with higher predicted long-run volatility receive lower weights, mirroring the logic of professional risk parity funds such as Bridgewater's All Weather.

Method B — Shape Trading (full curve):

$$\text{score}_i = \frac{|\hat{\beta}_{0,i}|}{\text{std}(\beta_0)} + \frac{|\hat{\beta}_{1,i}|}{\text{std}(\beta_1)} + \frac{|\hat{\beta}_{2,i}|}{\text{std}(\beta_2)}, \qquad w_i^B \propto \frac{1}{\text{score}_i} \tag{6}$$

Stocks with irregular or elevated curve geometry across all three dimensions are penalized.

Method C — Momentum with predicted volatility:

$$\text{mom}_i = \frac{P_{i,t} - P_{i,t-63}}{P_{i,t-63}}, \qquad w_i^C \propto \max\left(\frac{\text{mom}_i}{\hat{\beta}_{0,i}},\, 0\right) \tag{7}$$

Stocks in positive trend and expected to be calm receive higher weights, combining the momentum premium [Jegadeesh and Titman, 1993] with forward-looking volatility.

All methods apply weight constraints $w_i \in [0.05, 0.25]$ enforced via SLSQP. When average predicted volatility exceeds a threshold, total portfolio exposure is reduced to 70% of capital.

4 Results and Reflection

4.1 Full-Period Performance (2018–2026)

The backtest covers 2,109 trading days with 10 basis points transaction costs per unit of daily turnover. A final equity of 1.0 represents the initial capital.

Table 1: Backtesting results, January 2018 – May 2026

Strategy	**Sharpe**	**Max DD**	**CAGR**	**Calmar**
Equal Weighted	0.698	−31.94%	12.12%	0.380
Historical Risk Parity	0.629	−30.13%	9.82%	0.326
Method A (XGBoost)	0.715	−27.92%	11.66%	0.417
Method B (XGBoost)	0.616	−33.53%	10.52%	0.314
Method C (XGBoost)	0.842	−32.81%	15.16%	0.462
Method A (ARMA)	0.734	−28.84%	12.14%	0.421

4.2 Stress Period Analysis

Table 2: Performance during critical market periods

Strategy	**Return**	**Vol (ann.)**	**Sharpe**	**Max DD**
COVID crash: March 1 – April 30, 2020				
Equal Weighted	−3.43%	74.24%	0.506	−26.99%
Historical Risk Parity	−3.63%	68.47%	0.478	−25.04%
Method A (XGBoost)	−0.00%	68.04%	0.799	−22.96%
Method B (XGBoost)	−3.63%	77.14%	0.492	−28.14%
Method C (XGBoost)	−4.93%	73.27%	0.350	−26.68%
Method A (ARMA)	+0.57%	69.75%	0.840	−24.18%
Bear market: September 1 – December 31, 2022				
Equal Weighted	+6.41%	22.29%	0.930	−12.35%
Historical Risk Parity	+4.85%	19.89%	0.854	−10.81%
Method A (XGBoost)	+7.35%	19.78%	1.240	−10.33%
Method B (XGBoost)	+5.54%	21.75%	0.868	−12.27%
Method C (XGBoost)	+7.98%	19.89%	1.192	−11.25%
Method A (ARMA)	+7.74%	20.78%	1.244	−10.30%

4.3 Discussion of Research Questions

RQ2 — ML forecasting vs. Historical Risk Parity. Both Method A variants outperform Historical Risk Parity (Sharpe 0.629) on all metrics, with lower drawdowns and higher CAGR. This confirms the central thesis: predicting volatility forward adds genuine value over backward-looking estimation. The stress period results are particularly compelling — during the COVID crash, Method A (XGBoost) suffered essentially zero loss while equal weight lost 3.43%.

RQ1 — XGBoost vs. ARMA. Method A with ARMA achieves a higher Sharpe (0.734 vs. 0.715) at a fraction of the transaction costs (0.0276 vs. 0.0955). This is the most interesting result of the project. Nelson-Siegel betas exhibit strong linear autocorrelation, which ARMA captures as effectively as XGBoost — while generating smoother weight transitions and lower turnover. This is consistent with Gu, Kelly and Xiu [2020], who demonstrate that ML models do not universally dominate simpler benchmarks on financial time series. More complexity is not always better.

RQ3 — Curve shape vs. level alone. Method B (Sharpe 0.616) underperforms Method A (0.715). Incorporating slope and curvature hurts rather than helps: these parameters are noisier, harder to forecast accurately, and their prediction errors propagate into suboptimal weight allocation. The level β_0 alone captures the most relevant information for portfolio construction.

RQ4 — Momentum enhancement. Method C achieves the highest Sharpe (0.842) and CAGR (15.16%), confirming that combining trend with predicted volatility is a powerful strategy. The cost is higher drawdown (-32.81%) and transaction costs, reflecting the more aggressive rebalancing induced by momentum signals. For a risk-tolerant investor seeking growth, Method C dominates. For a risk-conscious investor seeking capital preservation, Method A (ARMA) is superior.

5 Lessons Learned

Honest backtesting requires strict discipline. Walk-forward validation is not a technical detail — it is the foundation of credible empirical finance. Any normalization, scaling, or parameter estimation that uses future data invalidates the entire backtest. We enforced per-day normalization strictly, ensuring every prediction uses only information available at that point in time.

Simpler models can and do win. The finding that ARMA outperforms XGBoost on Method A is not a failure of the project — it is its most valuable insight. In quantitative finance, the bias-variance tradeoff favors simpler models on noisy daily data. Transaction cost awareness further reinforces this: a model that generates smoother weights is cheaper to operate, and net-of-cost performance is what matters in practice.

Volatility forecasting adds real value over historical estimation. All three active methods that use predicted volatility outperform Historical Risk Parity, which only looks backward. The margin is largest during stress periods, precisely when good risk management matters most. This validates the core premise of the project.

Agentic development is a genuine workflow improvement. Automating the pipeline

through agents — rather than running scripts manually — reduced errors, enforced consistency, and created a transparent audit trail on GitHub. The Trading Agent's automatic Pull Request is not just a course requirement; it is a pattern that scales to real production systems.

AI-assisted development requires critical judgment. Generative AI tools accelerated every phase of the project, from methodology design to code generation to documentation. The real skill was not prompting — it was evaluating outputs critically, identifying subtle bugs, and knowing when to push back. The course's emphasis on *process over product* reflects an important truth: in 2026, anyone can generate code. Understanding what it does and why is the differentiating capability.

References

References

Bollerslev, T. (1986). Generalized autoregressive conditional heteroscedasticity. *Journal of Econometrics*, 31(3), 307–327.

Carr, P. and Wu, L. (2009). Variance risk premiums. *Review of Financial Studies*, 22(3), 1311–1341.

Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *KDD 2016*, 785–794.

Diebold, F.X. and Li, C. (2006). Forecasting the term structure of government bond yields. *Journal of Econometrics*, 130(2), 337–364.

Engle, R.F. (1982). Autoregressive conditional heteroscedasticity. *Econometrica*, 50(4), 987–1007.

Fama, E.F. (1970). Efficient capital markets. *Journal of Finance*, 25(2), 383–417.

Gu, S., Kelly, B. and Xiu, D. (2020). Empirical asset pricing via machine learning. *Review of Financial Studies*, 33(5), 2223–2273.

Jegadeesh, N. and Titman, S. (1993). Returns to buying winners and selling losers. *Journal of Finance*, 48(1), 65–91.

Maillard, S., Roncalli, T. and Teiletche, J. (2010). Equally weighted risk contribution portfolios. *Journal of Portfolio Management*, 36(4), 60–70.

Nelson, C.R. and Siegel, A.F. (1987). Parsimonious modeling of yield curves. *Journal of Business*, 60(4), 473–489.